

### Soal Ganjil

1. Jelaskan apa yang dimaksud dengan race!
2. Apa itu critical section dalam konteks sinkronisasi proses?
3. Sebutkan dan jelaskan tiga syarat yang harus dipenuhi dalam penyelesaian masalah critical section!
4. Bagaimana cara menghindari kondisi Circular Wait agar deadlock tidak terjadi dalam suatu sistem?
5. Sebuah sistem memiliki dua proses yang masing-masing membutuhkan akses ke dua sumber daya (misalnya, printer dan scanner). Jelaskan bagaimana deadlock dapat terjadi dalam kasus ini dan bagaimana cara mencegahnya!

### Jawaban

1. Race condition terjadi ketika dua atau lebih proses/thread mengakses data bersama secara bersamaan, sehingga hasil akhir bergantung pada urutan eksekusi dan timing.
2. Critical section adalah bagian kode di mana proses/thread mengakses atau memodifikasi sumber daya bersama, yang harus diakses secara eksklusif untuk menghindari race condition.
3.
  1. Mutual Exclusion: Hanya satu proses yang boleh berada di critical section pada satu waktu.
  2. Progress: Proses yang ingin masuk critical section harus diizinkan jika tidak ada proses lain di dalamnya.
  3. Bounded Waiting: Ada batas berapa kali proses lain bisa masuk ke critical section sebelum satu proses tertentu mendapat giliran.
4. Untuk menghindari Circular Wait: tetapkan urutan hierarkis dalam mengakses sumber daya, alokasikan semua sumber daya sekaligus, atau terapkan sistem yang bisa melepaskan sumber daya jika tidak bisa mendapatkan semua yang dibutuhkan.
5. Deadlock terjadi saat Proses A memegang scanner dan menunggu printer, sementara Proses B memegang printer dan menunggu scanner. Pencegahan: tetapkan urutan akses yang konsisten (misalnya selalu akses printer dulu baru scanner), atau minta kedua sumber daya sekaligus, atau terapkan timeout untuk melepaskan sumber daya jika proses menunggu terlalu lama.

### Soal Praktik

Buatlah sebuah program menggunakan kode c tanpa menggunakan mutex yang melakukan proses iterasi hingga 10000000 menggunakan 2 buah threading dan tampilkan output dari nilai akhir counternya. Coba run program tersebut 2/3 kali dan jelaskan apa yang terjadi.

Langkah :

1. Buat kode program.c menggunakan perintah **nano program.c** di linux
2. Jalankan perintah **gcc program.c -o program -pthread**
3. Jalankan program dengan perintah **./program**

Masukkan screenshoot kode disini



```
GNU nano 7.2                                program.c *
#include <stdio.h>
#include <pthread.h>

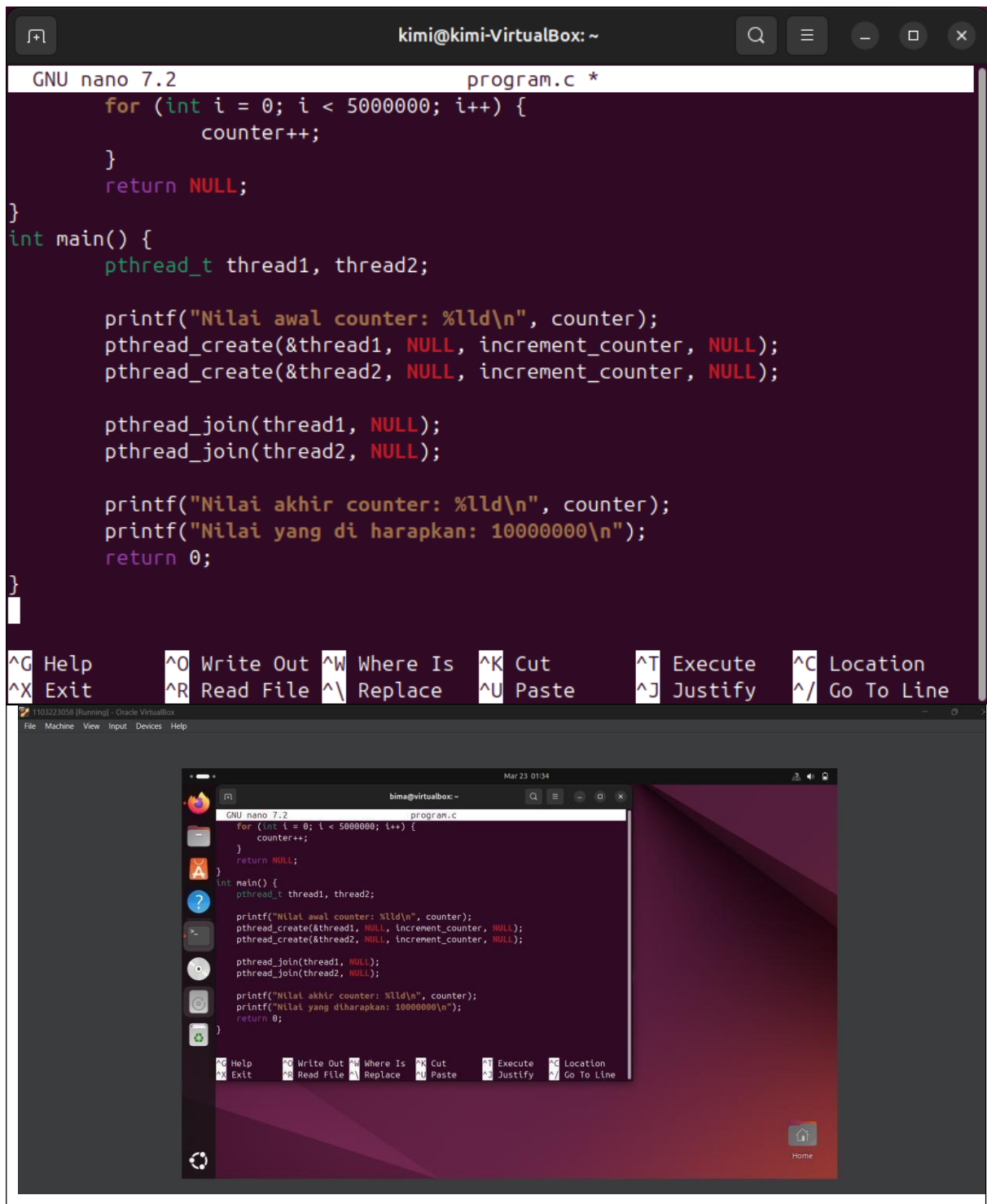
long long counter = 0;
void* increment_counter(void* arg) {
    for (int i = 0; i < 5000000; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t thread1, thread2;

    printf("Nilai awal counter: %lld\n", counter);
    pthread_create(&thread1, NULL, increment_counter, NULL);
    pthread_create(&thread2, NULL, increment_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
}
```

^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location  
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^\_ Go To Line



Masukkan screenshoot output disini

```
kimi@kimi-VirtualBox:~$ nano program.c
kimi@kimi-VirtualBox:~$ nano program.c
kimi@kimi-VirtualBox:~$ gcc program.c -o program -pthread
kimi@kimi-VirtualBox:~$ ./program
Nilai awal counter: 0
Nilai akhir counter: 8047112
Nilai yang di harapkan: 10000000
kimi@kimi-VirtualBox:~$ ./program
Nilai awal counter: 0
Nilai akhir counter: 6783288
Nilai yang di harapkan: 10000000
kimi@kimi-VirtualBox:~$ ./program
Nilai awal counter: 0
Nilai akhir counter: 6906471
Nilai yang di harapkan: 10000000
kimi@kimi-VirtualBox:~$
```

Penjelasan : Jadi program ini adalah fenomena yang disebut race condition, yang dimana kondisi dua thread mengakses dan memodifikasi data yang sama secara bersamaan tanpa proteksi dari mutex

